

Group 25, Xinrong Zhu (xz875), Adam Gelb (ag2384), Isabella Calderon (cac542)

Robot Design and Strategy Overview

Our robot's mechanical design consists of the shell that houses the components of the robot, the outer 3D printed chassis, the motors and wheels, and the arms that extend from the chassis. The shell of the robot is the given sheet metal body. From this, there is a platform above which houses the arduino and breadboard. There is a TCS3200 color sensor mounted to the bottom of the metal body, which allows us to determine color through wavelength received. There are two motors attached to the side of the metal body, fastened with small M3 screws. These motors have wheels attached, with rubber bands along the outside of the wheels to increase friction and maneuverability. There is a small spherical wheel located at the back of the metal body, which allows for the robot to stay level and drive effectively. The 3D printed chassis attaches to the metal body, as the spacers used to fasten the platform above fits into holes in the chassis. This chassis was designed to protect the robot and give the cubes an area to sit in while the robot drives. Attached to the chassis are two arms, loosely fastened to the chassis with screws. We want these arms to swing freely, so that they fall down when the round begins as if they started in the down position the robot would be larger than 8 inches. We did not want to use servos because they only add to the complexity, and the problem can be solved in software.

Electrically, The robot uses an Arduino Uno, breadboard, 2 H-bridges, a TCS3200 Color Sensor, two motors, and an external 6 volt and 9 volt battery to run. We note that we decided to exclude the QTI black and white sensors from our robot as we used the color sensor to know when to turn. The Arduino Uno was powered by 9 volts, and was used to control the motors. On our breadboard, we had 2 H-bridges, which would take an input signal from the Arduino, and output a signal to the motors to make them spin. The Arduino took both software input and an input from the color sensor. The color sensor was in the 20 percent, red filter mode. Whenever a color change was sensed, the Arduino received a signal and executed code. All components were connected to a common ground. The H-bridges and motors were powered by 6 volts, and the color sensor was connected to 5 volts from the Arduino.

In software, the robot uses interrupts to measure the output signal of the TCS3200 color sensor and detect changes in surface color. The sensor is configured to use the red filter at 20% scaling, and the robot determines color by measuring the period of the sensor's output signal. At the start of the game, the robot records an initial period value corresponding to the starting surface color. It also briefly moves back and forth to deploy the sweeper arms. During operation, the robot continuously measures the current signal period and compares it to the initial baseline. If the difference exceeds

a predefined threshold, the robot interprets this as a color change. When this occurs, the robot stops, briefly reverses, and then turns slightly to the right before continuing forward. If no significant color change is detected, the robot continues driving forward. All motor movements are controlled through functions that set the appropriate outputs to the H-bridges. In an ideal scenario, the robot drives in a loop within the starting color area, allowing it to collect cubes along its path.

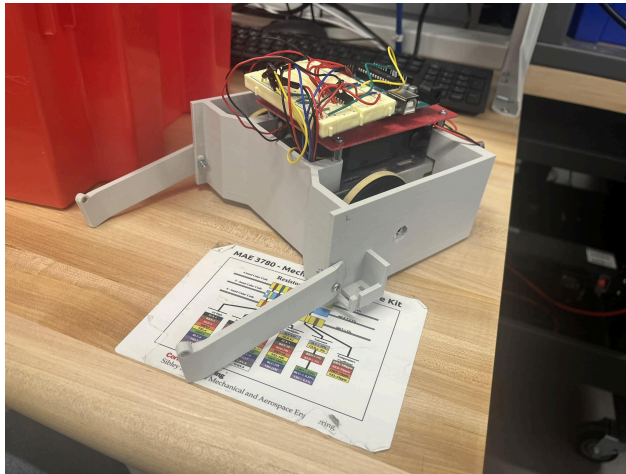


Figure 1: Final Robot on Competition Day

Design Process Reflection

Our original design was pretty basic. We decided that it would be best to have some sort of enclosure around our robot to capture cubes. This turned out to be the 3d print that we designed, although there needed to be some modifications to the chassis after it was printed. In terms of the milestones, we were very successful in completing them in a timely manner. The first milestone was completed on day 1 with a successful presentation and flowchart. Milestone 2 was completed the next day, with the robot being built and in its most bare bone configuration. We used information and documentation from lecture and lab to successfully create the robot and the code to drive. Milestone 3 was completed about a week later after we implemented the color and black and white sensors on the robot. After this milestone, we decided that the black and white sensors were not necessary for our design and how our code worked, so we removed them. Milestone 4 was completed after our chassis was built and modified so it would fit the robot. The originally printed chassis and sweeper arms, as shown in Appendix C, would have had servo motors on each side of the robot to hold the sweeper arms out of the way until the start of the competition. We realized after printing everything out that this was vastly overcomplicating the process, and the original chassis would not fit. To fix this, we cut the back off the chassis, and had the arms lightly screwed into the 3d printed frame. They would start in the up position, and

the robot would quickly move back and forth at the start of the competition to lower them.

Competition Analysis

On competition day, our robot unfortunately struggled due to a minute difference between the practice boards we used in the lab and the final competition boards in Duffield Atrium; while the practice boards in the lab had edges flush to the tabletop, the competition boards were slightly raised off the ground. Because of this oversight, during the competition our robot would always get too close to the edge and then one of the robot's arms would get stuck on the edge, immobilizing the robot for the remainder of the round. We tried to remedy this mid-competition by adding hot glue under the arms to prevent them from sinking down so low that they got stuck on the edge of the board, but this unfortunately was not very successful. Mainly because of this issue, our robot lost every round of the competition.

Conclusions

One of the major takeaways from this project was the importance of testing systems in scenarios as similar to their intended use as possible. If we had realized there was a difference in the boards we could have adjusted the code for our robot to prevent it from getting stuck on the edges. Another major conclusion we reached was that with this project, less was not necessarily more. Our design, especially of the mechanical components, was very basic with the idea that unnecessary complexity would only add opportunities for failure. While this philosophy sounded good in principle, in practice our robot was not able to compete well against more mechanically complex robots.

Appendix A: Bill of Materials

Part	Quantity	Total Cost
Chassis, 3D printed	1	70.26
Right sweeper arm, 3D printed	1	4.40
Left sweeper arm, 3D printed	1	4.40

Appendix B: Circuit Diagram

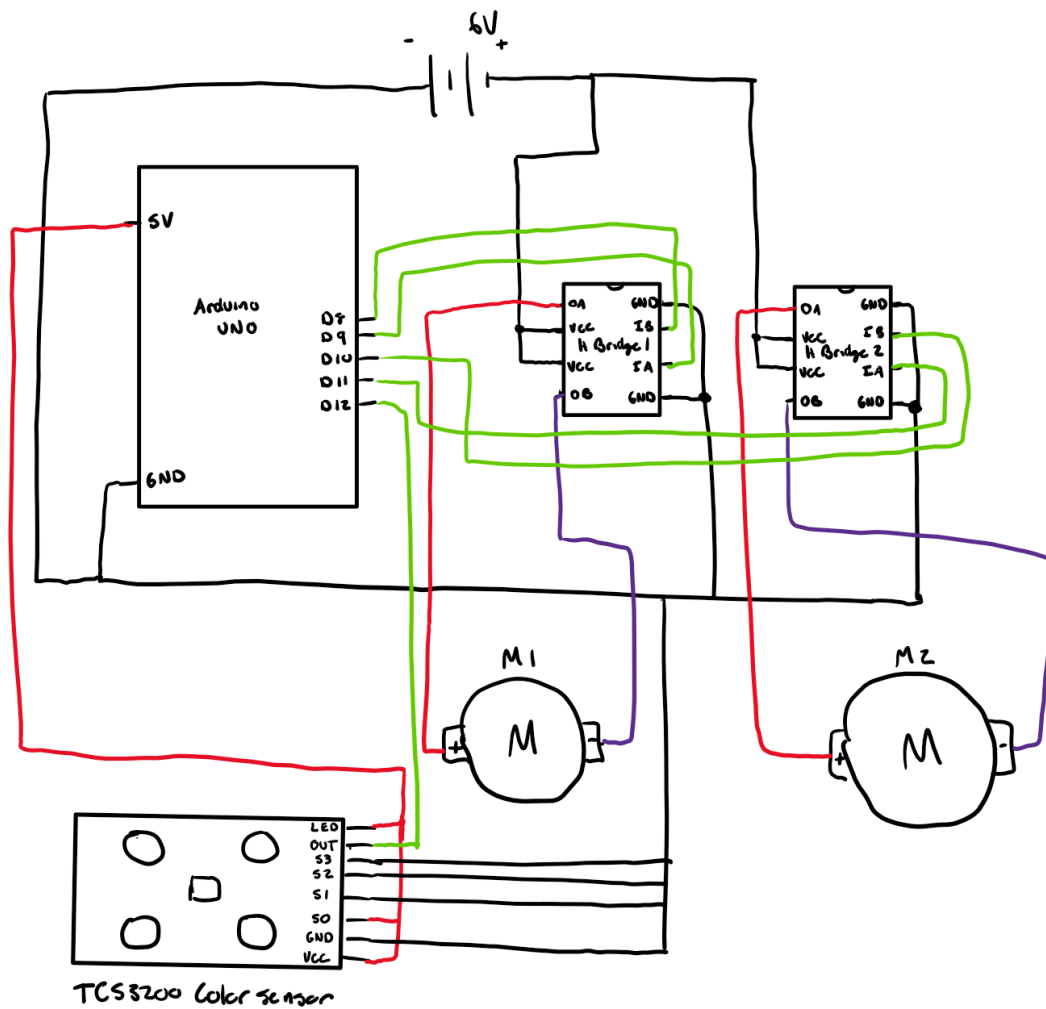


Figure 2: Circuit diagram for our robot. Consists of an Arduino Uno, 2 H-bridges, 2 motors, and a TCS3200 color sensor.

Appendix C: CAD Files and Drawings

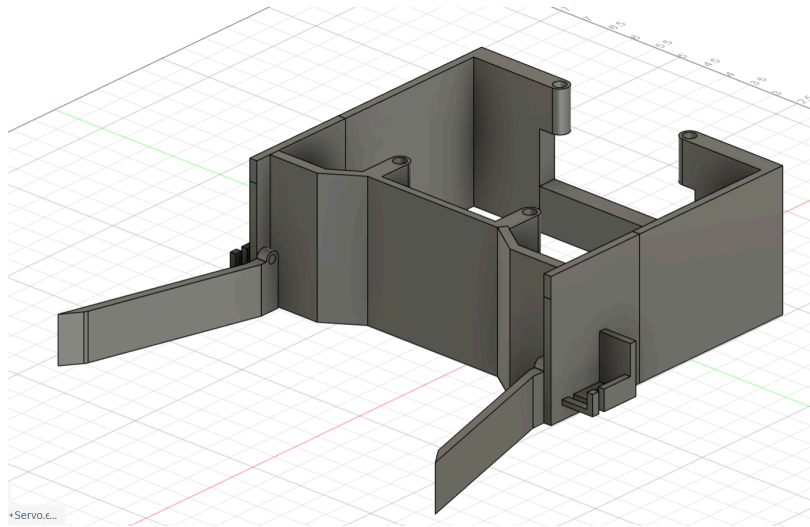


Figure 3: Fusion 360 CAD file for chassis, left arm, and right arm as originally designed and printed

Object name: MAE 3780 CAD Chassis.stl
Size: 137.787 x 195.267 x 66.04 mm
Volume: 184706 mm³
Triangles: 1692

$$\text{Volume} = 184.71 \text{ cm}^3$$

$$\text{Weight} = \text{Volume} \times 0.75 \times 1.25 \text{ g/cm}^3$$

$$\text{Weight} = 173.16 \text{ grams}$$

Object name: MAE 3780 CAD Right Sweeper.stl
Size: 98.4163 x 29.3651 x 26.3649 mm
Volume: 9061.55 mm³
Triangles: 212

$$\text{Volume} = 9.061 \text{ cm}^3$$

$$\text{Weight} = \text{Volume} \times 0.75 \times 1.25 \text{ g/cm}^3$$

$$\text{Weight} = 8.50 \text{ grams}$$

Object name: MAE 3780 CAD Left Sweeper.stl
Size: 98.4163 x 29.3651 x 26.3649 mm
Volume: 9061.55 mm³
Triangles: 212

$$Volume = 9.061 \text{ cm}^3$$

$$Weight = Volume \times 0.75 \times 1.25 \text{ g/cm}^3$$

$$Weight = 8.50 \text{ grams}$$

Appendix D: Flowchart

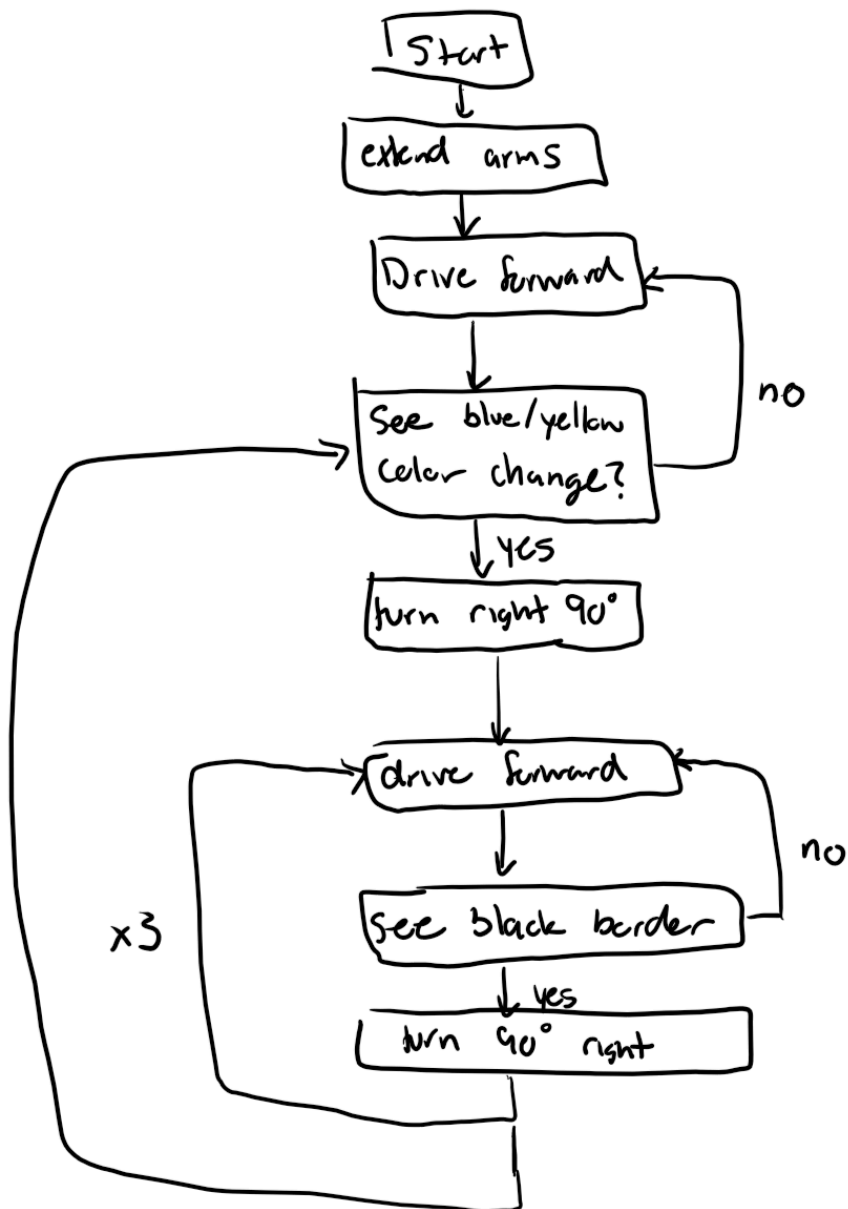


Figure 4: Initial flowchart for our robot. If a color change is seen, the robot turns right, and continues to do this until the round ends.

Appendix E: Code

```
// --- Global Variables ---

// "period" : stores the value of the output wave period (not necessarily in seconds)
volatile int period = 0;

// "timer" : stores the value of TIMER1
volatile int timer = 0;

// --- ISR: an interrupt function that resets and reads the timer value

ISR(PCINT0_vect)
{
    // resets the timer to zero on a rising edge,
    if (PINB & 0b00010000) { //PINB is turned high (rising edge)
        TCNT1 = 0;
    } else { // and stores the timer value in a variable ("timer") on a falling edge (or vice versa).
        timer = TCNT1;
        period = timer;
    }
}

// --- initColor: initializes the interrupt and timer

void initColor()
{
    // Initialize I/O pins
    DDRB = DDRB & 0b11101111; //pin 12 as input for sensor
    // Initialize my pin change interrupt
    PCICR = 0b00000001;
    PCMSK0 = 0b00010000;
    // Initialize your timer
    TCCR1A = 0b00000000; //set timer1 to normal mode
    TCCR1B = 0b00000001; //set prescaler or timer1 to 1
    TCNT1 = 0; //set timer to 0
}

// --- getColor: calculates the period of the sensor output wave and returns it as a variable
```



```

int getColor()
{

    // Enable the specific bit for your Pin Change Interrupt
    PCMSK0 = 0b00010000;

    // Add a short delay (5 ~ 10 milliseconds)
    _delay_ms(10);

    // Disable the specific bit for your pin change interrupt
    // (to prevent further interrupts until you call getColor again)
    PCMSK0 = 0b00000000;
    // Return the period in units of time
    return period * 0.0625 * 2; // 0.0625us per tick, times 2 for full period
}

int main(void)
{
    Serial.begin(9600);
    // call initColor function
    initColor();
    // enable global interrupts
    sei();
    // outputs for motors
    DDRB |= 0b00001111;

    // PD2 (left) & PD3 (right) = line sensor input
    DDRD &= 0b11110011;

    // get initial period number to compare to when color changed later
    int initialPeriod = getColor();

    // drive back and forward to deploy sweeper arms
    drive_backward();
    _delay_ms(100);
    stop();
    drive_forward();
    _delay_ms(100);
    stop();
    drive_backward();
    _delay_ms(100);
}

```

```

stop();

while(1)
{
    // call getColor function to get current color
    int currentPeriod = getColor();

    if ((abs(currentPeriod-initialPeriod) > 95)){ //if detect different color
        // stop any motion and drive backwards slight, and then turn right slightly to drive around the color change border
        stop();
        drive_backward();
        _delay_ms(90);
        stop();
        turn_right();
        _delay_ms(150);
        stop();
        drive_forward();

    } else { // if no color change is detected, drive forward
        drive_forward();
    }

    // add short driving delay to prevent instant motion
    _delay_ms(100);
}

}

// drivetrain functions to drive forward, backward, right, left, and stop
void drive_forward(){
    PORTB=PORTB | 0b00000110;
}

void drive_backward(){
    PORTB=PORTB | 0b00001001;
}

void turn_right(){
    PORTB=PORTB | 0b00000101;
}

void turn_left(){
    PORTB=PORTB | 0b00001010;
}

void stop(){

```

```
PORTB &= 0b11110000;  
}
```